

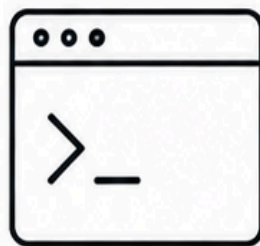
---

# DEVOPS COMMAND BATTLE

PROOF OF CONCEPT (POC)

---

AI-Assisted Development · GitHub Copilot Exploration · Vibe Coding Experiment



*An AI-assisted experiment to build an interactive web application  
for learning DevOps and Linux commands.*

---

Prepared by

**Joyal B Biju**

---



May 2026

# 1. Executive Summary

This document presents a Proof of Concept (POC) for DevOps Command Battle, a browser-based application developed as an experiment in AI-assisted software development using GitHub Copilot and modern "vibe coding" practices.

Unlike traditional software projects that focus primarily on product delivery, this POC was initiated as a learning initiative. As a cloud engineer rather than a professional software developer, the primary objective was to gain practical experience with GitHub Copilot, understand AI-assisted coding workflows, and explore rapid prototyping through human-AI collaboration.

The outcome was a fully functional web application deployed publicly via GitHub Pages, demonstrating the feasibility of AI-supported problem solving for infrastructure professionals.

# 2. Project Motivation

The emergence of AI coding assistants like GitHub Copilot is reshaping the landscape of software development. While developers have quickly adopted these tools, many cloud and infrastructure professionals have limited hands-on exposure to these workflows.

This project was created to answer a fundamental question: Can a cloud engineer with limited software development experience use GitHub Copilot and vibe coding techniques to build a functional application?

The purpose was to move beyond theoretical study and understand Copilot's capabilities through real-world implementation.

### 3. Product Overview: DevOps Command Battle

DevOps Command Battle is an interactive, browser-based application designed around DevOps and Linux command challenges delivered through a gamified interface.

#### Core Features

- **Command challenges:** Scenario-based technical tasks.
- **Interactive gameplay:** Real-time feedback and engagement.
- **Browser-based execution:** No local installation required.
- **Scoring system:** Logic for tracking progression and performance.
- **Public deployment:** Hosted via GitHub Pages for easy access.

### 4. Architecture & Workflow

The application follows a lightweight frontend architecture where all logic executes directly within the user's browser, intentionally avoiding backend dependencies to maintain focus on rapid experimentation.

#### System Architecture

- **User Layer:** Browser interface for interaction.
- **Interface Layer:** HTML5 and CSS3 for structure and visual design.
- **Logic Engine:** JavaScript-based game mechanics and challenge processing.
- **Persistence:** Browser LocalStorage for state and progress tracking.

#### Operational Workflow

1. User opens the application via the GitHub Pages URL.
2. The interface loads the core gameplay components.
3. JavaScript logic generates technical challenges.
4. User inputs commands/responses.
5. System validates responses and updates scores.
6. LocalStorage preserves the state for future sessions.

## 5. Development Stack & AI Workflow

The project utilized a modern, lightweight technology stack to facilitate rapid iteration and testing.

| Component    | Technology        |
|--------------|-------------------|
| Structure    | HTML5             |
| Styling      | CSS3              |
| Logic        | JavaScript (ES6+) |
| Persistence  | LocalStorage      |
| AI Assistant | GitHub Copilot    |
| Deployment   | GitHub Pages      |

### AI-Assisted "Vibe Coding" Workflow

Development followed an iterative cycle: **Idea → Prompt → Copilot Suggestion → Human Review → Modification → Testing → Final Code.**

GitHub Copilot acted as a productivity accelerator by:

- Generating boilerplate and starter code.
- Suggesting UI components and styling.
- Assisting with complex JavaScript logic.
- Accelerating the debugging process during testing.

## 6. Testing & Validation

Functional testing was conducted to ensure stable behavior across different user scenarios.

| Action           | Expected Result              | Actual Result |
|------------------|------------------------------|---------------|
| Open application | Interface loads successfully | Successful ▾  |
| Start challenge  | Challenge is displayed       | Successful ▾  |
| Submit answer    | Logic processes response     | Successful ▾  |
| Correct answer   | Score updates correctly      | Successful ▾  |
| Incorrect answer | Error feedback generated     | Successful ▾  |
| Refresh browser  | Game state is retained       | Successful ▾  |
| Public access    | URL is accessible globally   | Successful ▾  |

## 7. Results & Key Learnings

### Outcomes

- Successfully deployed a functional browser application.
- Established a repeatable GitHub deployment workflow.
- Gained practical mastery of AI-assisted coding and "vibe coding" methodologies.

### Challenges

- Bridging the gap between infrastructure logic and frontend engineering.
- Refining AI suggestions to meet specific UI requirements.
- Managing state persistence within a serverless frontend environment.

## 8. Conclusion

DevOps Command Battle successfully validated GitHub Copilot as a powerful learning and development companion for cloud engineers. By reducing the entry barrier to software development, these tools allow infrastructure professionals to rapidly prototype ideas into functional tools.

Further exploration into combining AI-assisted development with automated cloud infrastructure (Terraform/EKS) is recommended for future POC iterations.

## Project References

- **GitHub Repository:** <https://github.com/JoyalBBiju/devops-command-battle>
- **Live Deployment:** <https://joyalbbiju.github.io/devops-command-battle>